React JS Interview Question Bank for AI Grading

This document contains core and advanced React JS interview questions designed for an automated AI grading system. Each question includes an **Ideal Answer**, **Key Keywords** (for automatic transcription mapping), and a **3-Tier Grading Rubric** (to guide the AI evaluator).

1. Core Concepts: The Virtual DOM & Reconciliation

- **Category:** Core Concepts
- **Difficulty:** Medium

Question

Explain how the Virtual DOM works in React. How does the "Reconciliation" process take place?

Ideal Answer

The Virtual DOM is a lightweight, in-memory JavaScript representation of the Real DOM. When a component's state or props change, React builds a new Virtual DOM tree. It then compares this new tree with the previous Virtual DOM tree using a highly optimized diffing algorithm with a heuristic complexity of $O(n)$.

This comparison process is called "Reconciliation". Once the differences are identified, React batches the updates and applies only the minimal necessary changes to the Real DOM, reducing heavy layout recalculations and rendering cycles in the browser.

Key Keywords to Track

- Virtual DOM
- Real DOM
- Diffing algorithm
- Reconciliation
- Batch update
- State/Props change
- $O(n)$ complexity (or linear time complexity)

Grading Rubric

- **Score 5 (Excellent):** The candidate explains the Virtual DOM clearly, articulates the diffing algorithm and the Reconciliation process, notes the $O(n)$ heuristic complexity, and details how batching updates to the Real DOM optimizes browser performance.
- **Score 3 (Good):** The candidate understands that the Virtual DOM is a copy of the Real

DOM used to find differences, but lacks deep detail on the mechanics of the Reconciliation/diffing algorithm or batching updates.
- **Score 1 (Poor):** The candidate only knows that the Virtual DOM is "faster" than the Real DOM, but cannot explain the technical reasons or the underlying synchronization process.

2. Hooks: useEffect Dependency Configurations & Cleanup

- **Category:** Hooks
- **Difficulty:** Medium

Question

Differentiate the behavior of useEffect when:

1. No dependency array is passed.
2. An empty dependency array [] is passed.
3. A dependency array with variables [dep] is passed.

How do you perform cleanup inside useEffect and why is it important?

Ideal Answer

- **No dependency array:** The callback runs after every single render of the component.
- **Empty array []:** The callback runs only once, immediately after the component mounts.
- **With variables [dep]:** The callback runs on the initial mount and subsequently only when any value in the dependency array changes.

To clean up effects (e.g., to prevent memory leaks, clear timeouts, or remove event listeners), you return a function from the useEffect callback. This cleanup function runs before the component unmounts and before running the effect again on subsequent renders.

Key Keywords to Track

- Dependency array
- Mount / Unmount
- Cleanup function
- Memory leak
- Return statement (or returning a function)
- Render cycle

Grading Rubric

- **Score 5 (Excellent):** The candidate correctly differentiates all three dependency scenarios and provides a clear explanation of how the cleanup function works (when it executes and its crucial role in preventing memory leaks or cleaning up side-effects).
- **Score 3 (Good):** The candidate knows the difference between the three dependency array configurations but is vague about how the cleanup function operates or why it is

essential to prevent memory leaks.
- **Score 1 (Poor):** The candidate confuses the dependency array behaviors or fails to explain how to write or use a cleanup function.

3. Performance Optimization: useMemo vs useCallback

- **Category:** Performance Optimization
- **Difficulty:** Hard

Question

What are useMemo and useCallback used for? What are the potential performance downsides of overusing these two Hooks?

Ideal Answer

- useMemo memoizes (caches) the computed *value* of an expensive calculation between re-renders, recalculating only when dependencies change.
- useCallback memoizes the actual *function definition* to maintain referential equality across renders. This is highly useful when passing callbacks to memoized child components (using React.memo).

Overusing them can degrade performance because both Hooks require overhead: allocating memory for the dependency array, executing shallow comparisons on every render, and keeping references in memory. For trivial calculations or simple functions, this overhead outweighs the optimization benefits.

Key Keywords to Track

- Memoization
- useMemo
- useCallback
- Referential equality
- Shallow comparison
- Re-render overhead
- React.memo

Grading Rubric

- **Score 5 (Excellent):** The candidate clearly distinguishes between memoizing a computed value (useMemo) vs. a function definition (useCallback). They deeply analyze the performance overhead (memory overhead, shallow comparison cycles) indicating why blind optimization actually harms performance.
- **Score 3 (Good):** The candidate can define both Hooks and describe when to use them for performance, but has a limited or superficial understanding of why overusing them could

negatively impact the application.
- **Score 1 (Poor):** The candidate does not understand the difference between the two Hooks or wrongly believes that wrapping everything in useMemo and useCallback automatically makes the application faster.

4. State Management: Prop Drilling & Architectural Solutions

- **Category:** State Management
- **Difficulty:** Medium

Question

What is "Prop Drilling"? What are the common strategies to avoid it in React, and when would you prefer one approach over the others?

Ideal Answer

Prop Drilling refers to the practice of passing data (props) through multiple nested child components that do not actually need the data themselves, solely to deliver it to a deeply nested target component.

Strategies to resolve this include:

1. **Component Composition:** Passing child elements directly or rendering components at a higher level.
2. **Context API:** Best for lightweight, global-like state that updates infrequently (e.g., UI themes, current user locale).
3. **External Global State Management Libraries:** (such as Redux, Zustand, Recoil) best for complex, fast-changing, or large-scale states.

Composition is preferred for simple UI layout structures, Context for low-frequency global updates, and global libraries for high-performance and complex business logic architectures.

Key Keywords to Track

- Prop Drilling
- Component Composition
- Context API
- Global State Management
- Zustand / Redux / Recoil
- Re-render overhead
- Nested components

Grading Rubric

- **Score 5 (Excellent):** The candidate defines Prop Drilling perfectly, provides at least three

alternative solutions (Composition, Context, State Managers), and explains the technical trade-offs (e.g., Context causing unnecessary re-renders of consuming subtrees vs. state management libraries minimizing updates).
- **Score 3 (Good):** The candidate understands the concept and suggests using Context API or Redux, but fails to mention Component Composition or cannot provide concrete criteria on when to pick one over the other.
- **Score 1 (Poor):** The candidate cannot define Prop Drilling or relies entirely on manually passing props down through nested levels without knowing alternative solutions.

5. Advanced Architecture: React Server Components (RSC) vs Client Components

- **Category:** Advanced Architecture
- **Difficulty:** Hard

Question

What is the core difference between React Server Components (RSC) and Client Components? When should you use each type?

Ideal Answer

- **React Server Components (RSC):** Render entirely on the server. The output sent to the client is a serialized JSON-like structure (not raw HTML), requiring zero client-side JavaScript bundle size for those components. RSCs cannot use interactive hooks (like useState, useEffect) or browser-only APIs.
- **Client Components:** (declared with 'use client') render primarily on the client, allow complete interactivity, and can use state hooks, effects, and browser APIs.

Use RSCs by default for data fetching, static layouts, and heavy dependencies to optimize initial page loads and SEO. Use Client Components for interactive elements like forms, buttons, inputs, and real-time UI components.

Key Keywords to Track

- React Server Components
- Client Components
- "use client" directive
- Zero bundle size
- Data fetching
- Hydration
- Interactivity
- Server-side rendering (SSR)

Grading Rubric

- **Score 5 (Excellent):** The candidate analyzes the differences thoroughly, including build/bundle size impact, hydration mechanics, execution environments, and data fetching advantages. They offer distinct, realistic architectural boundaries for using each type.
- **Score 3 (Good):** The candidate identifies that Client Components use 'use client' and handle interactivity, whereas RSCs handle server tasks, but cannot explain the bundle size savings or the technical serialization/hydration relationship.
- **Score 1 (Poor):** The candidate is unaware of React Server Components or confuses them entirely with traditional Server-Side Rendering (SSR).